

Mini-TP 7 — Room et l'offline-first

Feuille de l'etudiant

Nom : RASETRATINJANAHARY Zotina

ETU : ETU002597

Etape 1 - Lire et predire (AVANT de lancer)

J'ai lu Donnees.kt : l'Entity Produit, le DAO avec 3 TODO, et la Database. J'ai aussi lu ProduitsViewModel.kt et MainActivity.kt. J'ai predit sans lancer.

Prediction	Votre reponse
P1 — Au tout premier lancement, avant vos TODO : qu'affichera l'écran (liste, stock total, puces de mode) ?	La liste affichera tous les 5 produits triés par nom (Vanille, Café, Girofle, Litchi, Poivre). Le stock total affichera "Stock total : (TODO 3 — non branché)" car l'agrégat n'est pas encore branché. Les puces de mode seront affichées mais ne feront rien car les modes ne sont pas branchés.
P2 — Vous fermez COMPLÈTEMENT l'application (pas seulement l'écran), puis vous la rouvrez. Les produits sont-ils encore là ? Et le mode d'affichage choisi ?	Les produits sont encore là car la base Room persiste sur le téléphone — elle n'est pas détruite quand l'application ferme. En revanche, le mode d'affichage choisi NE sera pas gardé car le MutableStateFlow du mode vit en mémoire et est recréé à zéro au redémarrage de l'application.

Etape 2 - Les trois requêtes

TODO 1 — TRI :

```
@Query("SELECT * FROM produits ORDER BY prixKg IS NULL, prixKg DESC")
fun parPrixDecroissant(): Flow<List<Produit>>
```

`prixKg IS NULL` renvoie TRUE (1) pour les NULL et FALSE (0) pour les non-NULL. `ORDER BY` trie en ordre croissant, donc 0 (non-NULL) avant 1 (NULL). Les NULL vont donc en dernier.

Question : pourquoi `prixKg IS NULL` en premier critère fait-il descendre les NULL en bas ?

Parce que `prixKg IS NULL` renvoie 1 pour les NULL et 0 pour les autres. `ORDER BY` trie par ordre croissant, donc les 0 (non-NULL) viennent avant les 1 (NULL). La clause suivante trie ensuite les non-NULL par prix décroissant.

TODO 2 — FILTRE :

```
@Query("SELECT * FROM produits WHERE stockKg > :seuilKg")
fun stockSuperieurA(seuilKg: Double): Flow<List<Produit>>
```

Le paramètre `seuilKg` est passé avec la syntaxe `:seuilKg`.

TODO 3 — AGRÉGAT :

```
@Query("SELECT SUM(stockKg) FROM produits")
fun stockTotal(): Flow<Double?>
```

Le résultat peut être null car si la table est vide, SUM retourne NULL. C'est pour ça que le type est `Flow<Double?>` et pas `Flow<Double>`.

Verification :

- [x] TODO 1 — tri par prix décroissant, NULL en dernier
- [x] TODO 2 — filtre par seuil de stock (paramètre)
- [x] TODO 3 — agrégat SUM du stock total

Etape 3 - Brancher et observer

J'ai décommenté le bloc ÉTAPE 3 dans `ProduitsViewModel.kt` :

- `dao.parPrixDecroissant()` branché sur le mode `PRIX_DECROISSANT`
- `dao.stockSuperieurA(10.0)` branché sur le mode `STOCK_SUFFISANT`
- `dao.stockTotal()` affiché dans le stock total

Observation : dans le tri par prix décroissant, le litchi se place EN DERNIER, après tous les produits qui ont un prix. C'est la clause `prixKg IS NULL` dans la requête qui décide cela — elle met les NULL en dernier dans l'ordre croissant.

Question de contrôle : le stock total affiché est **138.0 kg** ($18.5 + 42.0 + 15.0 + 55.0 + 7.5$).

Etape 4 - Voie ouverte : juger un test IA

J'ai soumis une requête du DAO à l'IA pour un test unitaire. J'ai trié la remarque.

Synthèse du tri en 3 lignes :

- Remarque la plus pertinente : l'IA a signalé qu'il faudrait tester le résultat de la requête avec des données connues, pas juste que la fonction ne plante pas.
- Remarque moins pertinente : l'IA a suggéré d'ajouter des tests d'intégration avec une vraie base — hors périmètre du jour, on teste les requêtes, pas la base entière.
- Le test de l'IA se contente probablement de constater que la fonction ne plante pas, ce qui n'a pas beaucoup de valeur — il faudrait vérifier le résultat réel de la requête.

Commentaires supplémentaires

Le projet compile avec les 3 requêtes Room. Room vérifie le SQL à la compilation (KSP) — une erreur de nom de colonne ne passerait pas. Le Flow ré-émet automatiquement quand la base change. La base persiste même si l'application est fermée complètement.

Le litchi (prix null) se place bien en dernier dans le tri par prix décroissant, confirmant que la requête fonctionne correctement.